

Darmstadt University of Applied Sciences

– Faculty of Computer Science –

AI driven evolution of penetration testing in software security

by

Henning Prien

Matriculation number: 1132644

First Examiner : Dominik Machold

DECLARATION

I hereby certify that I have written the present paper independently and have not used any sources other than those listed in the bibliography.

All passages that have been taken literally or in substance from published or unpublished sources are identified as such.

The drawings or illustrations in this work were created by myself or are provided with an appropriate source reference.

This work has not been submitted to any other examination authority in the same or a similar form.

Darmstadt, March 17, 2026

Henning Prien

ABSTRACT

Emerging threats such as advanced persistent threats, ransomware, and supply chain vulnerabilities require more dynamic and scalable security solutions than traditional methods can offer. This term paper investigates the potential of Artificial Intelligence (AI)-driven penetration testing, specifically through the use of Large Language Model (LLM)s to automate the process of penetration testing. Recent developments such as PentestGPT, PentestAgent, and ARTEMIS will be discussed.

PentestGPT and PentestAgent are capable of performing automated penetration tests in controlled environments. Tools like XBOW are able to make valuable findings in bug bounty programs. However, proprietary or niche technology stacks clearly mark their limits. Security experts note that while AI-driven tools excel at identifying and exploiting known vulnerabilities, they lack the lateral movement and contextual adaptability that human testers provide. Additionally, security risks such as data leakage through prompt injection and biases in training data pose further challenges.

Future research should focus on enhancing LLMs' ability to navigate proprietary systems, reducing false positives, and addressing security risks.

CONTENTS

I Thesis

1 Introduction 2
- 1.1 Problem Statement . . . 2
- 1.2 Goal of research . . . 2
- 1.3 Research Question and Hypotheses . . . 3
- 1.4 Structure . . . 3
- 1.5 Methodology . . . 3

2 State of Research and related Work 4

3 Measures of Software Security 5

4 Penetration Testing 7

5 AI driven penetration testing 8
- 5.1 Security Challenges . . . 10

6 Discussion 11

7 Conclusion and Outlook 12
- 7.1 LLMs are capable of performing automated penetration tests . 12
- 7.2 Hypotheses evaluation . . . 12
- 7.3 Functional limitations . . . 12
- 7.4 Future Work . . . 12

II Appendix

Bibliography 15

ABBREVIATIONS

SDLC Software Development Livecycle

SAST Static Application Security Test

DAST Dynamic Application Security Test

CIA Confidentiality Integrity Availability

AI Artificial Intelligence

LLM Large Language Model

CTF Capture The Flag

CLI Command Line Interface

RAG Retrieval-augmented generation

Part I

THESIS

1 INTRODUCTION

Software forms the backbone of modern society, supporting critical infrastructure such as communication networks, financial systems, and healthcare services. The security of these systems is a pressing concern, as vulnerabilities can lead to severe consequences, including financial losses, reputational harm, and even threats to human safety. Consequently, society has a significant indirect interest in software security due to its heavy reliance on digital infrastructure. [1]

Beyond societal concerns, software vendors also have compelling interests in providing secure software. On the one hand, companies can financially benefit from voluntarily disclosing their security measures. This transparency and proactive approach can positively influence their market value. [2]

On the other hand, software vendors are mandated to comply with various data protection laws and regulations. Non-compliance can result in severe penalties, including fines of up to 20,000,000€ limited to 4% of their total worldwide annual turnover. [3]

1.1 PROBLEM STATEMENT

Cyber security requirements need to be met while surrounded by a dynamically and fast evolving threat landscape. State and non-state actors drive a wide and diverse landscape of cyber security threats. Supply chain attacks, advanced persistent threats and vulnerabilities on the Internet of Things are just some cyber security threats that modern systems face. [4] More dynamic and scalable security solutions than traditional methods are needed to prevent data breaches and exploitable vulnerabilities. [5]

1.2 GOAL OF RESEARCH

To address growing difficulties of securing software systems, corporate vendors and developers can apply several security measures. The Software Development Livecycle (SDLC) describes multiple steps in which the development of secure software can be split up. One central part of the development of secure software is testing. [6] A highly effective approach of software testing is penetration testing [7]. Penetration testing used to be a manual and creative approach of dynamically testing for security vulnerabilities in software. However, traditional and manual methods of penetration testing are no longer sufficient to keep track of fast evolving threat landscapes and ever growing software systems. This raises the need for automated penetration

testing. [5] The goal of this term paper is to determine the extent to which penetration testing can be automated by leveraging LLMs.

1.3 RESEARCH QUESTION AND HYPOTHESES

The central research question addressed in this term paper is following: RQ: Are LLMs capable of performing automated penetration tests in corporate environments?

The hypotheses to answer this question is following: H: LLMs are capable of performing automated penetration tests on corporate and proprietary applications.

1.4 STRUCTURE

The main content of this term paper will consist of three parts. To emphasize the role of penetration testing in secure software development, the first step will differentiate between measures of software security. The second step will introduce penetration testing to answer the research question. The third step will examine AI-driven penetration testing. The findings of these three main parts will be carefully discussed. In the conclusion, all information will be summarized and synthesized. The hypotheses will also be evaluated based on these findings.

1.5 METHODOLOGY

A literature review will discuss past and ongoing research on the automation of penetration testing. Capabilities of earlier solutions will be compared to the current state of research. This term paper will partially be based on research papers that are still in pre-print or have not received much attention yet because they have been published recently. Findings of pre-print papers will be carefully discussed in this term paper to ensure quality. Because the technology of AI and especially LLMs is still evolving and receives broad attention, some aspects might change in the future. Special attention will be drawn to security experts' reports of experience. Corporate security experts actually working with AI driven penetration testing tools which is why their experiences cover the most recent evolutions. Goals for future research will be defined after consulting and discussing literature and experts' experiences.

2 STATE OF RESEARCH AND RELATED WORK

Penetration tests can be performed with three different knowledge levels: Black-Box penetration testing means that the penetration tester has no internal knowledge of the system under test. White-Box Testing means the Tester has complete knowledge of the internals and Grey-Box refers to partial knowledge of internal structures. [8]

Tools like BurpSuite, ZAP and SQLMap already implement automated penetration testing techniques through signature-driven Dynamic Application Security Test (DAST) scanning approaches. These scanners systematically map web applications, fuzz parameters with curated payloads, and detect injection vulnerabilities like XSS, SQLi, and command injection. However, these approaches are fundamentally constrained by their pattern-matching limitations that prevent the contextual reasoning, multi-step exploitation chains, and novel attack creativity required to identify complex business logic vulnerabilities and zero-day exploits. [9]

Earlier approaches to automate the process of penetration testing leveraged AI models trained through Reinforcement Learning [10]. Training environments for these models need to be high fidelity which makes the models inflexible. The lack of flexibility brought up difficulties scaling AI models trained through Reinforcement Learning to an production ready state. [11]

Leveraging LLMs for penetration testing shows better results and scalability. LLMs are able to apply attach patterns to applications more flexible. The project PentestGPT describes an agentic approach for orchestrating LLMs beneficially which further increases the performance. [12]

Little research has been conducted on the ability of LLMs to automatically perform penetration tests on corporate and proprietary applications. This term paper will investigate whether LLMs are able to perform automated penetration tests in corporate environments.

3 MEASURES OF SOFTWARE SECURITY

Multiple models and frameworks suggest steps that can be taken to develop secure software. They all describe different planning and implementation phases to cover every step of software engineering. [13]

The SDLC is one of these processes that allows the design, testing and implementation of secure software. It has received quite broad attention and many vendors use it to guideline their software development. The SDLC divides the software development into 6 steps:

1. Requirements and use cases
2. Design
3. Test plans
4. Code
5. Test results
6. Field Feedback

As this framework describes a life cycle, this process needs to be repeated when finished to implement the gathered feedback. The SDLC already highlights the importance of software testing although security measures can be taken in each step of the SDLC. [6]

A Systematic Mapping Study on Security Approaches in Secure Software Engineering was conducted in 2021. It revealed that many earlier conducted studies suggest to implement security measures in the design phase of the SDLC. The second most popular suggestion is to implement these measures through out the whole SDLC. Third, 23% suggest to implement security measures within the testing phase of the SDLC. [13] This clearly demonstrates the importance and popularity of implementing software security measures throughout the SDLC.

In the following, different software testing methods will be presented as the goal of this term paper is to evaluate how AI supports penetration testing. Afterwards the focus will be put to penetration testing as one of these testing techniques.

Before software runs even once, a Static Application Security Test (SAST) scanner is able to locate vulnerabilities in it. It can detect vulnerabilities in the source code itself as well as in byte or machine code. The weakness of these scanners is that they tend to have high false positive rates. [14]

Real attack scenarios like the exploitation of XSS and SQL injection are performed by DAST scanners. DAST scanners have no access to the source code of an application, nor to any runtime information like logs. [15] They

systematically search for standard vulnerabilities like insecure headers and unsecured endpoints. Because of this rather unadapted test strategy, they are not capable of finding logic errors or performing creative attacks.

4 PENETRATION TESTING

"[P][...]enetration testing is a detailed procedure to unravel [...] open doors in a system or network." [7] The technique of penetration testing is a creative process of understanding IT-systems and their detailed structure and functionality. Finding and documenting vulnerabilities in IT-infrastructure helps to secure them and enhance the safety of these systems.

DAST scanners test software for vulnerabilities by replaying known attack patterns. In comparison, penetration tests are more creative approaches. Testers adapt their testing plans based on system reactions and architectural details. Penetration tests simulate realistic attacks against a system under test, revealing in detail how real-world attackers could exploit existing vulnerabilities. This information is documented in a test report to inform IT infrastructure management and propose mitigation strategies. [7]

If a corporate vendor decides to conduct a penetration test on its system, this test can be performed either by internal employees or by a specialized third-party company. Either way the goal of finding exploitable vulnerabilities needs to be specified in detail. You can refer to the example of stealing a car. By a penetration test, a specialist can find out how hard it is to steal your car, but this appears not to be quite specific. Stealing a car usually refers to someone driving another's car without permission and not returning it. Does an unknown driving one's car without permission refers to stealing it when it is returned the next day? Does a teenager driving its parents car without parents permission refer to stealing it? The goal of finding out how hard it is to steal a car is simply not specific enough. For IT infrastructure a specific goal could for example be: Is an unauthorized party able to interfere with Confidentiality Integrity Availability (CIA) triad? [16]

In the next step it is important to consider the knowledge the penetration tester has about the system under test. Potential attackers can allocate massive resources in the form of time and money if the exploitation seems worthwhile. Through providing penetration testers with information about the system under test they can compensate for the disparity in resources. [16] Also penetration testers can be given partially internal access to simulate the scenario of already overcome first barriers by attackers. [16] Penetration testing falls into three categories based on knowledge level: Black-Box, Grey-Box, and White-Box penetration testing. [8]

5

AI DRIVEN PENETRATION TESTING

Earlier approaches to automate the process of penetration testing like ESA's PenBox leveraged Reinforcement Learning. PenBox has the capability to execute given actions like port scanning with NMap, SSH security testing with Hydra and exploitation of known vulnerabilities with Metasploit. Training landscapes were built to train PenBox to use the given actions beneficially. PenBox's training and test environment comprised 8 hosts in one network. In this rather small environment, PenBox was able to compromise vulnerable hosts with 33 actions taken while the minimum required actions would have been 4. [10]

Leveraging LLMs for penetration testing shows better results and scalability compared to Reinforcement Learning. Capture The Flag (CTF) Challenges from HackTheBox and VulnHub were used to evaluate the performance of GPT-4 without any technical additions. GPT-4 was prompted to advise the user to execute a specific task, such as running a Command Line Interface (CLI) command, whose output the user then prompts back to the LLM. Such a technically minimalistic setup proves that LLMs are capable of performing penetration tests as GPT-4 is able to solve 4 easy and 1 medium challenge. This benchmark was executed on 13 targets from HackTheBox and VulnHub covering OWASP Top 10 and categories easy, medium and hard. The available knowledge was entirely limited to the LLMs inherit domain knowledge. [12]

The process of penetration testing can be split into multiple tasks, which allows delegation to different LLM agents specialized in specific fields. The architecture of PentestGPT introduces three agents to delegate these tasks to. One agent is responsible for information condensing, one for reasoning, and one for concrete task planning. This agentic approach further improves the CTF challenge completion rate. Without the agentic approach, GPT-4 solved 4 easy and 1 medium challenge. With the agentic setup, it solved 6 easy and 2 medium challenges from the same set of challenges. The increase in CTF challenge completion rate points to the significance of the LLM orchestration. [12]

One limiting factor to higher CTF challenge completion is the lack of up-to-date domain-specific knowledge such as known vulnerabilities. The knowledge of an LLM is entangled within thousands of parameters, and retraining LLMs with up-to-date information is computationally very expensive. Retrieval-augmented generation (RAG), introduced by Lewis et al. [17], is a common approach to provide LLMs with knowledge not inherently built in. The additional information is stored as embeddings in a vector database, and the LLM is then provided with retrieved information from the RAG's vector database. [17]

PentestAgent improves upon PentestGPT's architecture by introducing domain-specific knowledge about known vulnerabilities and equipping the agentic setup with a RAG. This external knowledge base makes multiple vulnerability databases accessible for PentestAgent. Through multiple adjustments to the agentic setup and the newly introduced domain-specific knowledge, PentestAgent outperforms PentestGPT significantly when solving CTF challenges. 11 CTF challenges were randomly selected from HackTheBox (nine easy, one medium and one hard). PentestGPT solved 5 of these challenges (all easy), while PentestAgent solved 9 of these challenges (7 easy, 1 medium, and 1 hard). [18]

The success of PentestGPT and PentestAgent is not limited to small networks or single hosts. LLM-driven agents are also capable of performing penetration tests in real-world, full-scale networks. ARTEMIS is an agentic approach similar to PentestGPT and PentestAgent. However, it was optimized and tested in a real-world university network. Two versions of ARTEMIS were benchmarked against experienced human penetration testers. The first ARTEMIS version running entirely on GPT-5 identified 11 findings, of which 55% were valid. The second ARTEMIS version running on multiple different LLMs also identified 11 findings, with 82% being valid and having much higher severity. Both versions can compete with human penetration testers from this study. The ARTEMIS architects note that the agents scan systems broadly rather than moving laterally, as experienced human penetration testers follow approaches which typically yield findings with higher severities. [19]

Real world corporate applications are much more complicated and unknown than test environments like DVWA, OWASP Juice Shop, OWASP WebGoat and many CTF challenges. They have dynamic pages with JavaScript logic and proprietary, sometimes niche technology stacks. Automated testing tools must traverse the detailed architecture of the system under test. XBOW is an LLM-powered automated penetration testing tool that publicly claims to work with corporate applications and their challenges. XBOW can operate with and without access to the source code of the system under test. [20] Not much information about the detailed system structure is made public. XBOW's high position on the Hackerone leaderboard proves that it is capable of making valid findings in corporate infrastructure. [21] [22]

Corporate security experts who used XBOW on proprietary web applications shared their experiences for this term paper. To evaluate XBOW's performance, corporate security experts used a deliberately vulnerable web application based on proprietary technology as a benchmark system. The experts found that XBOW was unable to identify any vulnerabilities in this system. XBOW was provided with source code and network access to the application. According to the experts, XBOW is capable of finding vulnerabilities in other applications but tends to produce a high rate of false positives. The experts note that valid findings made by XBOW are generally low-hanging fruits from the perspective of experienced penetration testers.

Software vendors like Google implement LLM-powered penetration testing tools into software security processes. Google's open-source oss-fuzz, for example, is an LLM-powered fuzzing tool that has actively found many security flaws in open source software. [23]

5.1 SECURITY CHALLENGES

LLMs are trained on datasets that do not encompass the entirety of available information. The inherit bias from this training data can lead to unwanted treatment of actual vulnerabilities. A biased LLM can potentially overlook vulnerabilities. [24]

In the context of penetration testing, LLMs are often equipped with tools like web search. Prompts can be injected into the LLM's context through access to third-party webpages. Combined with the LLM holding confidential data, the risk of leaking this information to third-party websites rises.

6 DISCUSSION

Corporate networks span hundreds or thousands of hosts. Large corporate networks let Reinforcement Learning approaches reach their limits. The training process of Reinforcement Learning approaches, like PenBox, requires high-fidelity training environments. [11] Automating penetration testing with AI models trained through Reinforcement Learning has significant limitations. These approaches typically concentrate on exploiting known vulnerabilities through Metasploit and struggle with dynamic environments, making them unable to identify complex business logic vulnerabilities and zero-day exploits.

The vulnerabilities exploited by PentestGPT, PentestAgent, and ARTEMIS in benchmarks are nearly all based on known vulnerabilities or performed in controlled, unrealistic test environments. For corporate vendors, web application penetration testing is particularly interesting because web applications are highly exposed and widely targeted. Especially PentestAgent targets at exploiting known vulnerabilities instead of investigating architecture of web applications to find new vulnerabilities. For finding new vulnerabilities in unknown web applications these tools are not made and tested for.

PentestGPT was evaluated based on 13 CTF challenges, and PentestAgent's was evaluated based on 11 CTF challenges, which is even fewer. The selection process for these challenges is opaque. PentestGPT was evaluated on handpicked challenges, while PentestAgent was evaluated on randomly selected ones. Both selecting processes can influence the evaluation result.

Corporate LLM driven penetration testing tools like XBOW and oss-fuzz clearly show that LLMs are capable of finding security vulnerabilities in corporate software.

When proprietary technology stacks, including niche frameworks, XBOW reaches its limits in investigating proprietary codebases. Security experts who tested XBOW concluded that it lacks the lateral movement capabilities required to thoroughly investigate how applications function under test. This limitation is also observable in ARTEMIS's evaluation.

7 CONCLUSION AND OUTLOOK

7.1 LLMS ARE CAPABLE OF PERFORMING AUTOMATED PENETRATION TESTS

Tools like BurpSuite, ZAP and SQLMap already implement partially automated penetration testing techniques through signature-driven DAST scanning approaches. AI models trained through Reinforcement Learning are also used in these tools, mainly for classification.

With the rise of LLMs into public awareness, increasing amounts of research have focused on leveraging LLMs for automated penetration testing. Research has revealed that LLMs are already capable of performing penetration tests based on their inherent knowledge. Without any technical additions, publicly available LLMs can conduct penetration tests. By providing an LLM with tool usage capabilities, the process can be nearly fully automated. Additional domain-specific knowledge provided to the LLM improves the process further.

Corporate LLM-driven penetration testing tools like XBOW and oss-fuzz are capable of finding vulnerabilities in corporate tech stacks.

7.2 HYPOTHESES EVALUATION

The hypothesis that LLMs are capable of performing automated penetration tests on corporate and proprietary applications must be partially rejected due to functional limitations.

7.3 FUNCTIONAL LIMITATIONS

XBOW's high ranking on the Hackerone leaderboard clearly proves that LLM-powered tools are generally capable of finding vulnerabilities in real-world bug bounty programs. [21] [22] When working with proprietary tech stacks or applications built with niche frameworks, automated penetration tests leveraging LLMs reach their limits. Tests with proprietary tools like XBOW revealed that even highly sophisticated tools struggle with niche and proprietary tech stacks.

7.4 FUTURE WORK

With appropriate setup, LLMs can conduct end-to-end penetration tests. However, their vulnerability identification performance remains limited with proprietary or niche technology stacks. Future research should prioritize investigating the underlying causes of these limitations and developing strategies

to enhance AI-driven testing in corporate environments. Additionally, further research should address and mitigate the security challenges associated with these approaches.

Part II

APPENDIX

BIBLIOGRAPHY

[1] T. Given-Wilson, E. Baranov, and A. Legay, "Building user trust of critical digital technologies," in *2020 IEEE International Conference on Industrial Technology (ICIT)*, Feb. 2020, pp. 1199–1204. DOI: 10.1109/ICIT45562.2020.9067154.

[2] L. A. Gordon, M. P. Loeb, and T. Sohail, "Market value of voluntary disclosures concerning information security1," *Management Information Systems Quarterly*, vol. 34, no. 3, pp. 567–594, Sep. 2010, ISSN: 0276-7783. DOI: 10.2307/25750692.

[3] European Union, *Regulation (eu) 2016/679 of the european parliament and of the council of 27 april 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data, and repealing directive 95/46/ec (general data protection regulation)*, Official Journal of the European Union, L 119, 4 May 2016, pp. 1–88, 2016. [Online]. Available: https://eur-lex.europa.eu/legal-content/EN/TXT/PDF/?uri=CELEX:32016R0679.

[4] A. Adeyeri and H. Abroshan, "Geopolitical ramifications of cybersecurity threats: State responses and international cooperations in the digital warfare era," *Information*, vol. 15, no. 11, 2024, ISSN: 2078-2489. DOI: 10.3390/info15110682.

[5] Y. Stefinko, A. Piskozub, and A. Obshta, "Analysis of vulnerability characteristics for automated penetration testing," in *2024 IEEE 17th International Conference on Advanced Trends in Radioelectronics, Telecommunications and Computer Engineering (TCSET)*, Oct. 2024, pp. 449–453. DOI: 10.1109/TCSET64720.2024.10755620.

[6] B. Potter and G. McGraw, "Software security testing," *IEEE Security & Privacy*, vol. 2, no. 5, pp. 81–85, Sep. 2004, ISSN: 1558-4046. DOI: 10.1109/MSP.2004.84.

[7] P. Lachkov, L. Tawalbeh, and S. Bhatt, "Vulnerability assessment for applications security through penetration simulation and testing," *Journal of Web Engineering*, vol. 21, no. 7, pp. 2187–2208, Oct. 2022, ISSN: 1544-5976. DOI: 10.13052/jwe1540-9589.2178. [Online]. Available: https://ieeexplore.ieee.org/document/10251051.

[8] N. Rane and A. Qureshi, "Comparative analysis of automated scanning and manual penetration testing for enhanced cybersecurity," in *2024 12th International Symposium on Digital Forensics and Security (ISDFS)*, Apr. 2024, pp. 1–6. DOI: 10.1109/ISDFS60797.2024.10527240.

[9] T. A. Osman, J. R. Ragasa, H. Posesione, A. David, and M. Shakiba, "Comparative analysis of penetration testing tools in assessing web application vulnerabilities," in *2025 Interdisciplinary Conference on Electrics and Computer (INTCEC)*, 2025, pp. 1–6. DOI: 10.1109/INTCEC65580.2025.11256013.

[10] A. Confido, E. V. Ntagiou, and M. Wallum, "Reinforcing penetration testing using ai," in *2022 IEEE Aerospace Conference (AERO)*, 2022, pp. 1–15. DOI: 10.1109/AERO53065.2022.9843459.

[11] J. Schwartz and H. Kurniawati, "Autonomous penetration testing using reinforcement learning," *ArXiv*, vol. abs/1905.05965, 2019. [Online]. Available: https://api.semanticscholar.org/CorpusID:155090057.

[12] G. Deng, Y. Liu, V. Mayoral-Vilches, P. Liu, Y. Li, Y. Xu, T. Zhang, Y. Liu, M. Pinzger, and S. Rass, "PentestGPT: Evaluating and harnessing large language models for automated penetration testing," in *33rd USENIX Security Symposium (USENIX Security 24)*, Philadelphia, PA: USENIX Association, Aug. 2024, pp. 847–864, ISBN: 978-1-939133-44-1. [Online]. Available: https://www.usenix.org/conference/usenixsecurity24/presentation/deng.

[13] R. A. Khan, S. U. Khan, H. U. Khan, and M. Ilyas, "Systematic mapping study on security approaches in secure software engineering," *IEEE Access*, vol. 9, pp. 19 139–19 160, 2021, ISSN: 2169-3536. DOI: 10.1109/ACCESS.2021.3052311.

[14] D. Dalaq, K. F. Daya, A. Dalaq, M. N. Arefin, and M. K. Niazi, "A systematic literature review on static application security testing (sast) tools: Evaluation, benchmarks, challenges, and future directions," in *Proceedings of the 2025 29th International Conference on Evaluation and Assessment in Software Engineering Companion*, ser. EASE Companion '25, New York, NY, USA: Association for Computing Machinery, 2025, 162–168, ISBN: 9798400718328. DOI: 10.1145/3727967.3756838. [Online]. Available: https://doi.org/10.1145/3727967.3756838.

[15] R. Singh, M. Kumar Gupta, D. R. Patil, and S. Maruti Patil, "Analysis of web application vulnerabilities using dynamic application security testing," in *2024 IEEE 9th International Conference for Convergence in Technology (I2CT)*, 2024, pp. 1–6. DOI: 10.1109/I2CT61223.2024.10543484.

[16] M. Bishop, "About penetration testing," *IEEE Security & Privacy*, vol. 5, no. 6, pp. 84–87, 2007, ISSN: 1558-4046. DOI: 10.1109/MSP.2007.159.

[17] P. Lewis, E. Perez, A. Piktus, *et al.*, "Retrieval-augmented generation for knowledge-intensive nlp tasks," in *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, Eds., vol. 33, Curran Associates, Inc., 2020, pp. 9459–9474. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2020/file/6b493230205f780e1bc26945df7481e5-Paper.pdf.

[18] X. Shen, L. Wang, Z. Li, Y. Chen, W. Zhao, D. Sun, J. Wang, and W. Ruan, "Pentestagent: Incorporating llm agents to automated penetration testing," in *Proceedings of the 20th ACM Asia Conference on Computer and Communications Security*, ser. ASIA CCS '25, New York, NY, USA: Association for Computing Machinery, 2025, pp. 375–391, ISBN: 9798400714108. DOI: 10.1145/3708821.3733882. [Online]. Available: https://doi.org/10.1145/3708821.3733882.

[19] J. W. Lin, E. K. Jones, D. J. Jasper, *et al.*, *Comparing ai agents to cybersecurity professionals in real-world penetration testing*, 2025. arXiv: 2512.09882 [cs.AI]. [Online]. Available: https://arxiv.org/abs/2512.09882.

[20] *Xbow: Autonomous offensive security platform*, https://xbow.com/, Accessed: 2026-02-20, XBOW, 2026.

[21] HackerOne, *Leaderboard: All time reputation (q1 2026, us, web app, owasp a1)*, Accessed: 2026-02-23, 2026. [Online]. Available: https://hackerone.com/leaderboard/all_time_reputation?year=2026&quarter=1&owasp=a1&country=US&assetType=WEB_APP&tab=bbp&userTypeTab=collective.

[22] Arch1t3ct, *Behind the hype: Is xbow ai really the "game-changer" they claim it to be?* Accessed: 2026-03-06, 2025. [Online]. Available: https://godaccess.substack.com/p/behind-the-hype-is-xbow-ai-really-the-game-changer.

[23] A. Happe and J. Cito, *On the surprising efficacy of llms for penetration-testing*, 2025. arXiv: 2507.00829 [cs.CR]. [Online]. Available: https://arxiv.org/abs/2507.00829.

[24] E. Hilario, S. Azam, J. Sundaram, K. Imran Mohammed, and B. Shanmugam, "Generative ai for pentesting: The good, the bad, the ugly," *International Journal of Information Security*, vol. 23, no. 3, pp. 2075–2097, 2024, ISSN: 1615-5270. DOI: 10.1007/s10207-024-00835-x. [Online]. Available: https://doi.org/10.1007/s10207-024-00835-x.